

Can the GPU Accelerate Masked Template Matching?

A Cross-Platform Study in OpenCV

Tak Hogan
takhogan@gmail.com

May 2026

Abstract

Masked template matching, correlating a template against an image under a per-pixel weight mask, is a common operation that still works very well in user interface navigation when the target is a known image, but its OpenCV GPU paths receive much less attention than the unmasked case. In this paper we investigate: *can the GPU actually beat the CPU on this operation, and if so, when?* We add direct OpenCL kernels and CUDA kernels to OpenCV’s masked `matchTemplate` dispatcher and benchmark the modified build against a stock build on three machines covering Apple Silicon (macOS/OpenCL), an NVIDIA discrete GPU (CUDA + OpenCL on Linux), and a 2016-era Windows desktop with a discrete OpenCL device. The answer is that it depends on the machine. We saw a speedup on windows and linux environments, but a slowdown on macOS that could be due to Apple’s deprecated and unoptimized OpenCL stack. Our main contribution is a clean comparison of implementing masked template matching on different devices by compiling both versions on the same device to determine whether a GPU port of unmasked template matching actually pays off.

1 Introduction

Template matching computes, for every valid offset (x, y) , a similarity score between an image I and a template T of size $w \times h$. The *masked* variant additionally weights each pixel of the template by a mask M , so that for `TM_CCORR`,

$$R(x, y) = \sum_{u, v} M(u, v)^2 I(x+u, y+v) T(u, v),$$

with analogous expressions for `TM_SQDIFF` and the normed variants [1]. Masking matters whenever the template has irrelevant regions (transparent backgrounds, antialiased edges, partial occlusions); the motivating use case here is detecting small POI icons from google maps like cafés, parks, attractions inside map screenshots. A real world use case for this might be testing which amenities appear at different zoom levels on google maps as marketing visibility research.

The operation is, on paper, an excellent fit for GPUs: every output pixel is independent, the inner loop is a small dense dot product, and the input data fits

comfortably in modern GPU memory. However, the OpenCV library does not feature GPU acceleration options for the masked variant of template matching like it does the unmasked variant.

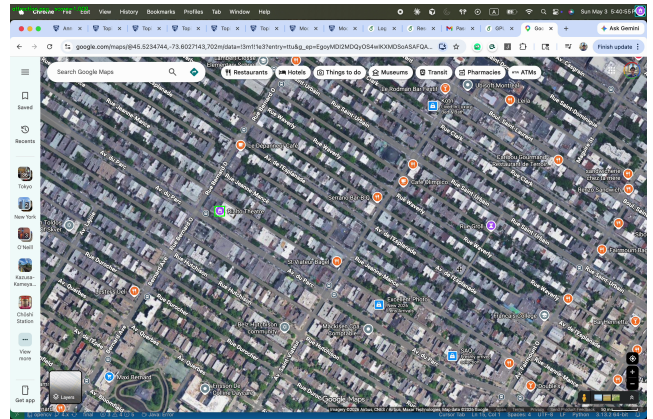


Figure 1: Example of masked template matching: the “attraction” POI icon (inset, top right) is located inside a map screenshot using `TM_CCORR_NORMED` with a hand-drawn alpha mask. The green box marks the best-scoring offset.

2 Background

OpenCV exposes three execution backends relevant here: `cv::Mat` (CPU, with SIMD and threading); `cv::UMat` (“Transparent API”) which dispatches to OpenCL when available; and `cv::cuda::GpuMat` which targets NVIDIA’s CUDA runtime directly. For unmasked `matchTemplate` all three backends have optimized implementations. For the *masked* case only the CPU implementations have been implemented by the library, the OpenCL path falls back to an FFT-based decomposition for all template sizes, and the CUDA path supports a subset of methods.

The FFT-based decomposition expresses the masked correlation as a sum of a few unmasked convolutions, each in $O(WH \log WH)$ through tiled DFTs. This has better theoretical runtime for large operations but has substantial fixed overhead from kernel build, multiple DFT round-trips, and intermediate complex-spectrum buffers. For small templates (the common case in icon detection), these overheads dominate. The question of whether to FFT or to evaluate the correlation directly

is therefore one implementation concern that we made sure to copy from the unmasked version of `matchTemplate`

3 What we changed

We modified `modules/imgproc/src/templmatch.cpp` to add two direct OpenCL kernels for masked `TM_CCORR` and `TM_SQDIFF` (and their normed variants):

- **Naive kernel.** One work-item per output pixel, computing the masked dot product directly from global memory with vectorized loads.
- **Tiled kernel.** A `_local`-memory variant that stages a slab of the image and reuses it across an output stripe.
- **CUDA Kernel** The CUDA version of the above two kernels using FFT or the naive approach depending on template size

A small dispatcher (`ocl_matchTemplateMask`) using builtin `opencv_opengl` methods routes very small templates to the naive kernel and falls through to the existing FFT path for everything else. The purpose of the modification is to give us a usable GPU path on every backend so the cross-platform comparison is honest.

4 Method

Benchmark harness. `masked_template_match.py` orchestrates the experiment by spawning a fresh subprocess for each `cv2` build (the unmodified “original” and our “new” build) so that no library state leaks between runs. The worker (`_bench_worker.py`) imports the requested `cv2`, executes each scenario across the requested backends (`cpu`, `umat/OpenCL`, `cuda`), runs 1 warm-up plus 10 timed iterations per (scenario, method, backend), and reports mean and standard deviation. After timing, the worker writes raw outputs to disk so the orchestrator can verify numerical agreement between the two builds (a sanity check that we have not silently broken correctness).

Scenarios. One synthetic case (a 2048×2048 random image with a 128×128 template under a circular mask) plus the Cartesian product of 5 map screenshots and 11 POI icon templates, each with a hand-drawn mask. We measure both `TM_CCORR_NORMED` and `TM_SQDIFF_NORMED`.

Hardware. Three machines:

- **Mac (2021 Apple M1 Silicon, macOS):** integrated GPU exposed via Apple’s deprecated OpenCL stack.
- **CUDA workstation (Linux):** NVIDIA discrete GPU, OpenCL via NVIDIA’s runtime, plus the `cv2.cuda` backend.

- **2016-era Windows desktop:** Dell OptiPlex 7020 SFF, Intel Core i5-4590 @ 3.30 GHz, 16 GB RAM, 250 GB SSD, AMD Radeon discrete GPU exposed via the vendor OpenCL runtime.

5 Results

Backend / scenario	CPU (ms)	GPU (ms)	Speedup
<i>Mac (Apple Silicon, OpenCL)</i>			
UMat, synthetic	613.7	939.5	0.65×
UMat, POI icons (avg)	523.4	1166.7	0.45×
<i>CUDA workstation (Linux)</i>			
UMat, synthetic	702.8	58.3	12.05×
UMat, POI icons (avg)	765.4	150.4	5.09×
CUDA, synthetic	702.8	30.9	22.75×
CUDA, POI icons (avg)	765.4	38.5	19.89×
<i>2016 Windows desktop (OpenCL)</i>			
UMat, synthetic	1000.8	635.4	1.57×
UMat, POI icons (avg)	883.3	531.1	1.66×

Table 1: End-to-end masked `matchTemplate` latency on the modified (“new”) OpenCV build, mean of 10 runs after one warm-up. Speedup is CPU / GPU on the same machine. Averaged across both `TM_CCORR_NORMED` and `TM_SQDIFF_NORMED`; the POI-icon row averages all 5×11 image/template combinations.

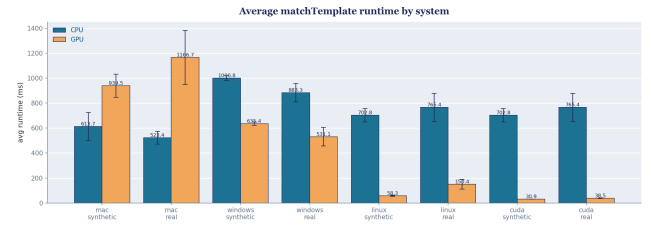


Figure 2: Average masked `matchTemplate` runtime by system, split into the synthetic case and the real POI-icon set, with CPU and GPU bars side by side. Lower is better. Error bars show one standard deviation across all timed runs in that group.

CUDA workstation: speedup. On the discrete NVIDIA GPU both the OpenCL UMat path and the dedicated `cv2.cuda` backend outperform the CPU baseline on the icon-detection workload. The operation is bandwidth-bound but very parallelizable, and the NVIDIA OpenCL and CUDA runtimes handles the small-template launch profile cleanly.

2016 Windows desktop: speedup. The older discrete GPU also beats the CPU, despite weaker absolute throughput. The driver is older and the device less powerful, but the workload is still parallel enough that GPU execution wins.

Mac (Apple Silicon): slowdown. Despite identical kernel source, the GPU path is *slower* than the CPU baseline on Apple Silicon. This may be

because Apple deprecated OpenCL in macOS 10.14 and the runtime now appears to poorly optimized. Per-phase logs show that `ocl::Kernel::create` alone takes a substantial fraction of the timed window, and `ocl::Queue::finish` latencies are noisy.

6 When and Why the GPU does (or does not) win

Several distinct factors decide whether masked `matchTemplate` benefits from GPU execution.

Driver/runtime maturity. The single biggest variable we found in our measurements seemed to be how optimized the OpenCL driver implementation were. NVIDIA's OpenCL was well tuned and the 2016 Windows vendor runtime was good given the specs of the device while Apple's deprecated OpenCL was not optimized and therefore run poorly despite good hardware.

Kernel launch and setup overhead. Masked `matchTemplate` as a single call does not run for very long milliseconds, not seconds. Anything that adds extra time per call (JIT-compiling kernels, allocating intermediate buffers, scheduling multiple DFT round-trips) competes with the work being accelerated. So this makes decisions like the FFT-vs-direct dispatch matter because for small templates the FFT path's setup is on the same order as the entire useful computation.

Memory transfer. The benchmark loads inputs once and runs ten matches. In a real icon-detection pipeline the same image may be matched against many templates, so reusing the same image is an appropriate comparison. If a new image needed to be created every time, the runtime profile would likely change.

Workload size. For the synthetic 2048×2048 scenario with 128×128 the GPU dominates everywhere except macOS. For the small POI templates the gap narrows but still macOS does not see the GPU speedup. Whether the GPU is worth the integration cost depends on the dominant workload, but still primarily on the device.

7 Conclusion

For masked template matching the answer to “does the GPU help?” is *yes, where the runtime is healthy*. On a modern NVIDIA discrete GPU and on a 2016-era Windows discrete GPU we measure speedups over the CPU baseline. On Apple Silicon, where OpenCL is deprecated and apparently no longer tuned for our access pattern, the same code does worse. For true optimization a metal port or staying on the CPU would be the right call on that platform. Future work includes a Metal backend for the masked path on macOS and an auto-tuned dispatch threshold so the cross-over between

direct and FFT evaluation is set per device rather than by a single hand-picked constant.

Reproducibility

The code can be run by simply calling `./setup.sh` followed by `./run.sh` on mac and linux from the <https://github.com/takhogan/opencv-masked-matchtemplate-benchmarking> repository or `./setup.ps1` followed by `./run.ps1` on windows. Note cmake must be in path and numpy must be installed locally with pip. OpenCV OpenCL source code changes can be found here: <https://github.com/takhogan/opencv/pull/1> OpenCV CUDA source code changes can be found here: https://github.com/takhogan/opencv_contrib/pull/1

References

- [1] OpenCV, *Object Detection — cv::matchTemplate*, OpenCV 4.x reference documentation. https://docs.opencv.org/4.x/df/dfb/group__imgproc__object.html#ga586ebfb0a7fb604b35a23d85391329be